

Daniel Toberman - 305706277

Doron Katzir - 307858894

Deep Learning – Exercise 1: Theoretical Part

Question 1

- Shape of input X : $X \in \mathbb{R}^{m \times 10}$.
- Hidden layer parameters: $W_h \in \mathbb{R}^{10 \times 50}$, $b_h \in \mathbb{R}^{50}$
- Output layer parameters: $W_o \in \mathbb{R}^{50 \times 3}$, $b_o \in \mathbb{R}^3$.
- Network output shape: $Y \in \mathbb{R}^{m \times 3}$.
- $Y = \text{ReLU}(\text{ReLU}(X \cdot W_h + b_h) \cdot W_o + b_o)$

Question 2

The network consists of three convolutional layers, each using 3×3 kernels, a stride of 2, and SAME padding. The input image has 3 channels (RGB), and the number of output feature maps for each layer is 100, 200, and 400, respectively. The total number of learnable parameters in a convolutional layer (including bias terms) is given by:

$$\text{Parameters} = (k_h \times k_w \times C_{in} \times C_{out}) + C_{out}$$

Where:

- k_h, k_w are the kernel height and width,
- C_{in} is the number of input channels, and
- C_{out} the number of output channels.

Parameters for each layer:

- **Conv1**: $3 \times 3 \times 3 \times 100 + 100 = 2,800$
- **Conv2**: $3 \times 3 \times 100 \times 200 + 200 = 180,200$
- **Conv3**: $3 \times 3 \times 200 \times 400 + 400 = 720,400$

Summing all layers:

Total parameters = $2,800 + 180,200 + 720,400 = 903,400$

Therefore, the entire network contains 903,400 learnable parameters. (The stride and padding affect the output spatial dimensions but do not change the number of learnable parameters).

Question 3

$$a. \frac{\partial f}{\partial \gamma} = \sum_{i=1}^m \frac{\partial f}{\partial y_i} \cdot \frac{\partial y_i}{\partial \gamma} = \sum_{i=1}^m \frac{\partial f}{\partial y_i} \cdot \hat{x}_i$$

(chain rule) $\left(\frac{\partial y_i}{\partial \gamma} = \hat{x}_i \right)$

$$b. \frac{\partial f}{\partial \beta} = \sum_{i=1}^m \frac{\partial f}{\partial y_i} \cdot \frac{\partial y_i}{\partial \beta} = \sum_{i=1}^m \frac{\partial f}{\partial y_i}$$

$\left(\frac{\partial y_i}{\partial \beta} = 1 \right)$

$$c. \frac{\partial f}{\partial \hat{x}_i} = \sum_{j=1}^m \frac{\partial f}{\partial y_j} \cdot \frac{\partial y_j}{\partial \hat{x}_i} = \frac{\partial f}{\partial y_i} \cdot \frac{\partial y_i}{\partial \hat{x}_i} = \frac{\partial f}{\partial y_i} \cdot 1$$

$\left(\frac{\partial y_j}{\partial \hat{x}_i} = \begin{cases} 0, & j \neq i \\ 1, & j = i \end{cases} \right)$

$$d. \frac{\partial f}{\partial \sigma^2} = \sum_{i=1}^m \frac{\partial f}{\partial \hat{x}_i} \cdot \frac{\partial \hat{x}_i}{\partial \sigma^2}$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}} \Rightarrow \frac{\partial \hat{x}_i}{\partial \sigma^2} = -\frac{1}{2} \cdot \frac{(x_i - \mu) \cdot (\sigma^2 + \varepsilon)^{-\frac{1}{2}}}{(\sigma^2 + \varepsilon)} = -\frac{1}{2} \cdot \frac{(x_i - \mu)}{(\sigma^2 + \varepsilon)^{\frac{3}{2}}}$$

$$\Rightarrow \frac{\partial f}{\partial \sigma^2} = -\frac{1}{2} \sum_{i=1}^m \frac{\partial f}{\partial \hat{x}_i} \cdot \frac{(x_i - \mu)}{(\sigma^2 + \varepsilon)^{\frac{3}{2}}}$$

$$e. \frac{\partial f}{\partial \mu} = \sum_{i=1}^m \underbrace{\frac{\partial f}{\partial \hat{x}_i} \cdot \frac{\partial \hat{x}_i}{\partial \mu}}_{(*)} + \underbrace{\frac{\partial f}{\partial \sigma^2} \cdot \frac{\partial \sigma^2}{\partial \mu}}_{(**)}$$

$$(*) : \frac{\partial \hat{x}_i}{\partial \mu} = -\frac{1}{\sqrt{\sigma^2 + \varepsilon}} \Rightarrow (*) = \frac{\partial f}{\partial \hat{x}_i} \cdot \left(-\frac{1}{\sqrt{\sigma^2 + \varepsilon}} \right)$$

$$(**) : \sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2 \Rightarrow \frac{\partial \sigma^2}{\partial \mu} = \frac{1}{m} \sum_{i=1}^m 2(x_i - \mu)(-1) = -\frac{2}{m} \sum_{i=1}^m (x_i - \mu) = 0$$

$\Rightarrow (**) = 0$

$$\Rightarrow \frac{\partial f}{\partial \mu} = \sum_{i=1}^m \frac{\partial f}{\partial \hat{x}_i} \cdot \left(-\frac{1}{\sqrt{\sigma^2 + \varepsilon}} \right)$$

$$f. \frac{\partial f}{\partial x_i} = \underbrace{\frac{\partial f}{\partial \hat{x}_i} \cdot \frac{\partial \hat{x}_i}{\partial x_i}}_{(*)} + \underbrace{\frac{\partial f}{\partial \mu} \cdot \frac{\partial \mu}{\partial x_i}}_{(**)} + \underbrace{\frac{\partial f}{\partial \sigma^2} \cdot \frac{\partial \sigma^2}{\partial x_i}}_{(***)}$$

$$(*) : \frac{\partial \hat{x}_i}{\partial x_i} = \frac{1}{\sqrt{\sigma^2 + \varepsilon}}$$

$$(**) : \frac{\partial f}{\partial \mu} = \sum_{i=1}^m \frac{\partial f}{\partial \hat{x}_i} \cdot \left(-\frac{1}{\sqrt{\sigma^2 + \varepsilon}} \right)$$

$$\frac{\partial \mu}{\partial x_i} = \frac{1}{m}$$

$$(***) : \frac{\partial f}{\partial \sigma^2} = -\frac{1}{2} \sum_{i=1}^m \frac{\partial f}{\partial \hat{x}_i} \cdot \frac{(x_i - \mu)}{(\sigma^2 + \varepsilon)^{\frac{3}{2}}}$$

$$\frac{\partial \sigma^2}{\partial x_i} = \frac{1}{m} \cdot 2 \cdot (x_i - \mu)$$

$$\Rightarrow \frac{\partial f}{\partial x_i} = \frac{\partial f}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma^2 + \varepsilon}} + \frac{1}{m} \cdot \sum_{j=1}^m \frac{\partial f}{\partial \hat{x}_j} \cdot \left(-\frac{1}{\sqrt{\sigma^2 + \varepsilon}} \right) + \left(-\frac{1}{2} \cdot \sum_{j=1}^m \frac{\partial f}{\partial \hat{x}_j} \cdot \frac{(x_j - \mu)}{(\sigma^2 + \varepsilon)^{\frac{3}{2}}} \right) \cdot \frac{2}{m} \cdot (x_i - \mu) =$$

$$= \frac{\partial f}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma^2 + \varepsilon}} - \frac{1}{m \sqrt{\sigma^2 + \varepsilon}} \cdot \sum_{j=1}^m \frac{\partial f}{\partial \hat{x}_j} - \frac{1}{m} \cdot \frac{1}{\sqrt{\sigma^2 + \varepsilon}} \cdot \underbrace{(x_i - \mu)}_{\hat{x}_i} \cdot \underbrace{\frac{1}{\sqrt{\sigma^2 + \varepsilon}} \cdot \sum_{j=1}^m \frac{\partial f}{\partial \hat{x}_j} \cdot \frac{(x_j - \mu)}{\sqrt{\sigma^2 + \varepsilon}}}_{\hat{x}_j} =$$

$$= \frac{\partial f}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma^2 + \varepsilon}} - \frac{1}{m \sqrt{\sigma^2 + \varepsilon}} \cdot \sum_{j=1}^m \frac{\partial f}{\partial \hat{x}_j} - \frac{1}{m \sqrt{\sigma^2 + \varepsilon}} \cdot \hat{x}_i \cdot \sum_{j=1}^m \frac{\partial f}{\partial \hat{x}_j} \cdot \hat{x}_j =$$

$$= \frac{1}{m \sqrt{\sigma^2 + \varepsilon}} \cdot \left(\frac{\partial f}{\partial \hat{x}_i} - \sum_{j=1}^m \frac{\partial f}{\partial \hat{x}_j} - \hat{x}_i \cdot \sum_{j=1}^m \frac{\partial f}{\partial \hat{x}_j} \cdot \hat{x}_j \right)$$

Practical Part

In this part we examine the performance of the LeNet-5 architecture on the Fashion-MNIST dataset, evaluating the following regularization techniques:

- **Baseline LeNet-5** - unmodified LeNet-5 architecture without any regularization components.
- **Dropout** - dropout layer ($p=0.5$) is added after the first fully-connected layer. During training, dropout randomly deactivates activations.. When evaluating accuracy, dropout is automatically disabled
- **Batch Normalization** - applied after both convolution layers.
- **Weight Decay** - (L2 regularization) penalizes model weights by adding a norm-based penalty to the loss function.

The implemented network uses the classical LeNet-5¹ structure, with a version fitted for 28×28 grayscale Fashion-MNIST. The input images are processed through the following stages:

Input:

1×28×28 grayscale image

Feature Extraction:

- Conv1:
 - 1 → 6, 5×5 kernel, stride 1, padding 2. Output shape: 6×28×28.
 - Optional BatchNorm after Conv1 in the BatchNorm configuration.
 - ReLU activation.
 - 2x2 Average pooling. Output shape: 6×14×14.
- Conv2:
 - 6 → 16, 5×5 kernel, stride 1. Output shape 16×10×10.
 - Optional BatchNorm after Conv2 in the BatchNorm configuration.
 - ReLU activation.
 - 2x2 Average pooling. Output shape 16×5×5.

After the second pooling operation, the spatial dimension reaches 5×5, yielding a flattened feature vector of length 400.

Classification Head:

- Fully connected: 400 → 120
- Fully connected: 120 → 84
- Fully connected: 84 → 10.

¹ Presented by *Yann LeCun et al. (1998): "Gradient-Based Learning Applied to Document Recognition"*.

The convolutional and fully-connected layer dimensions follow the original architecture, while ReLU activations are used instead of tanh, and averaging pooling replaces subsampling.

Batch normalization and dropout are parameterized as optional model components. Weight decay is supplied through the optimizer.

A supplementary script was developed to simplify model loading and evaluation. This script rebuilds a LeNet-5 instance in the corresponding configuration, loads its .pt weight file from the “checkpoints” folder, evaluates it on the full test set, and prints its final test accuracy. This enables immediate validation of trained models without re-training. All models in this project are trained using the Adam optimizer. In this implementation Adam is used with the following settings:

- learning rate: 1×10^{-3}
- β_1 : default (0.9)
- β_2 : default (0.999)
- ϵ : default ($1e-8$)

In the Weight Decay configuration, the Adam optimizer is additionally supplied with weight decay = 5×10^{-4} . Importantly, the optimizer, learning rate, batch size, and training schedule are kept identical across all four configurations to ensure a fair comparison. No learning-rate scheduling or early stopping is applied, to preserve clear comparability across different regularization strategies.

Results

Results are summarized in the Table 1 below: (also exported as a csv file from the main scripts):

Model	Final Train Accuracy (%)	Final Test Accuracy (%)
Baseline	97.82	90.47
Dropout	94.71	91.4
WeightDecay	95.35	90.74
BatchNorm	98.65	91.08

Table 1: Train & Test results on all four models

Figure 1-4 below show Train & Test accuracies plotted on the same graph for each model (separate graphs for Train & Test are presented in the appendix):

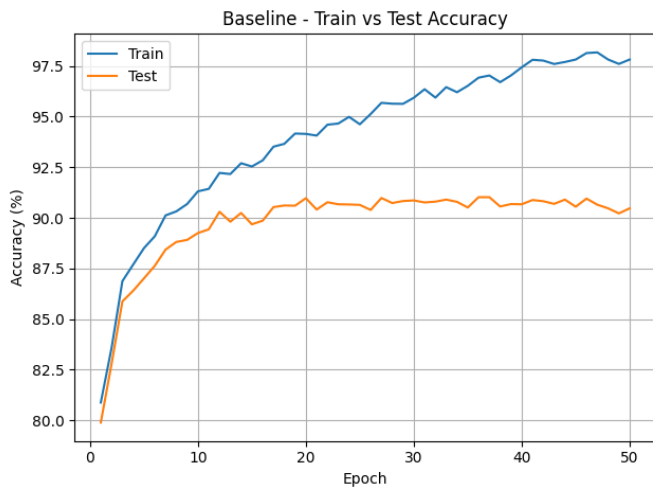


Figure 1: Convergence Graph for baseline LeNet-5

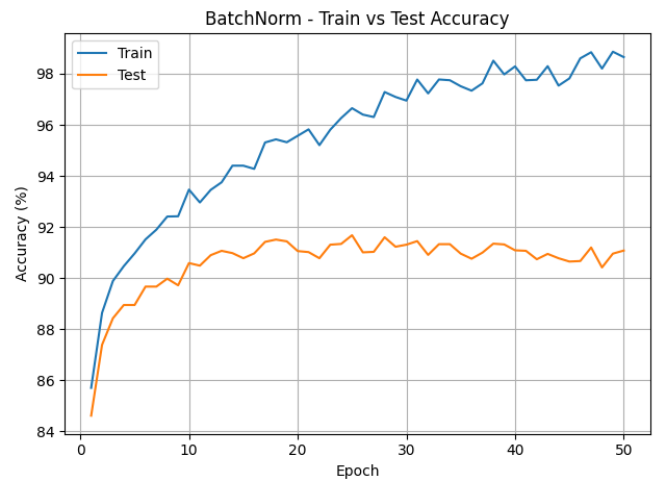


Figure 2: Convergence Graph for BatchNorm

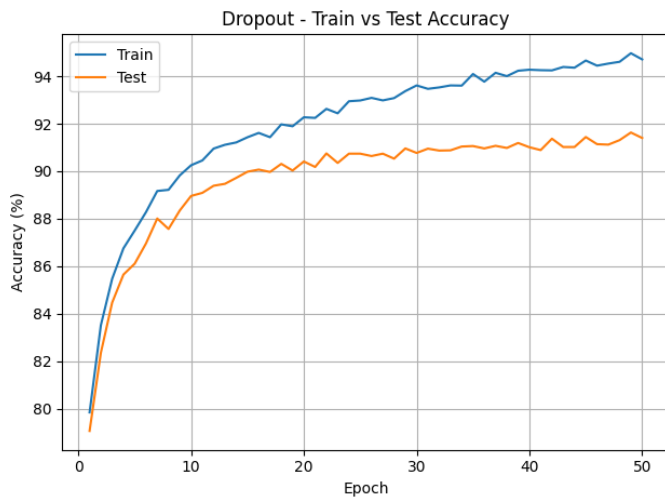


Figure 3: Convergence Graph for baseline Dropout

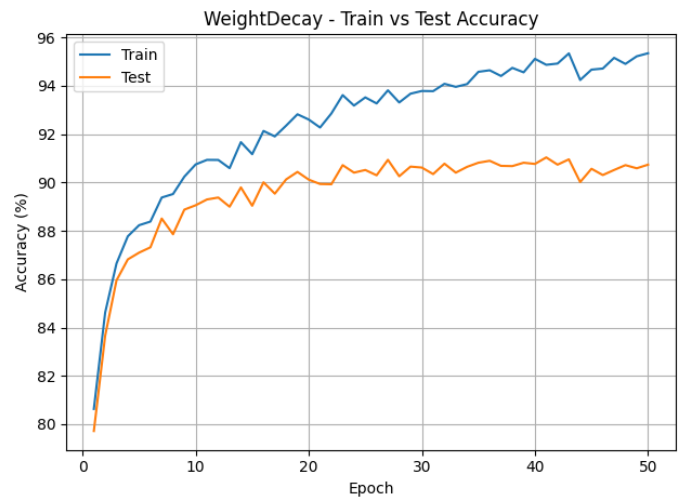


Figure 4: Convergence Graph for WeightDecay

Discussion

The baseline LeNet-5 model achieves high training accuracy and reaches a solid test accuracy, but shows clear overfitting: the training curve rises steadily toward ~98%, while the test curve plateaus around ~90–91%. The Dropout configuration reduces this overfitting effect. Its training accuracy remains significantly lower than the baseline across all epochs, while the test accuracy improves slightly. The train–test gap remains smaller, demonstrating the expected regularization effect.

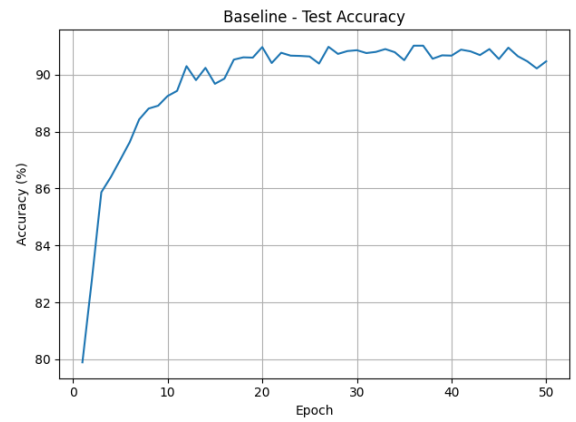
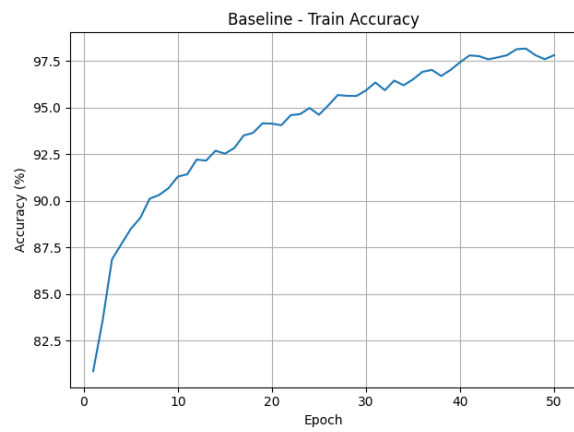
Weight Decay exhibits a similar improvement in generalization. Its training curve sits between the baseline and dropout curves, while its test performance stabilizes close to dropout. The reduced gap and smoother curve indicate more controlled weight growth.

Batch Normalization converges the fastest and reaches high accuracy. Although it does not remove overfitting, it consistently produces strong test results and improves optimization efficiency, particularly in the early epochs.

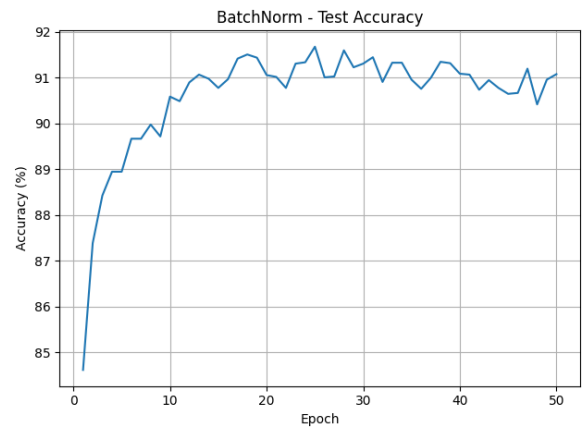
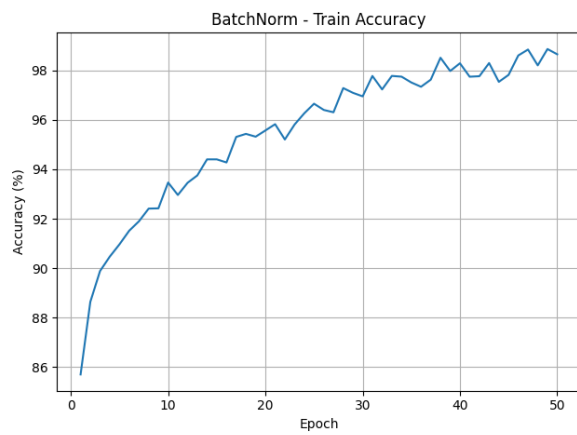
Overall, all three regularization methods outperform the baseline in terms of generalization, with BatchNorm converging the fastest, while Dropout producing slightly higher final test accuracy, and provides the smallest train-test gap.

Appendix

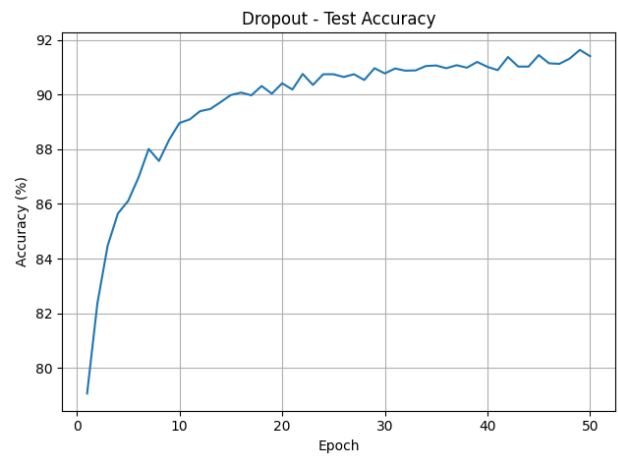
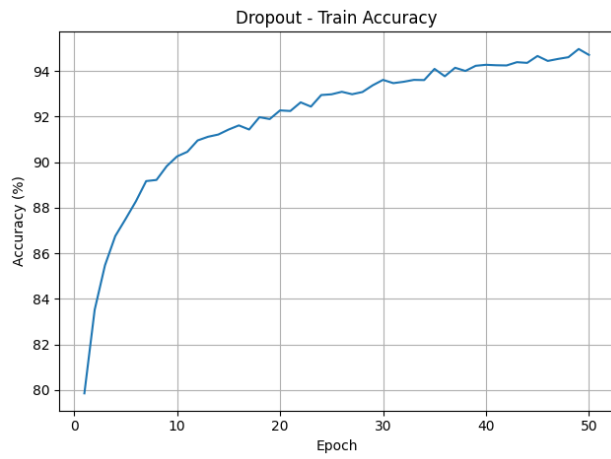
Convergence Graphs for baseline LeNet-5:



Convergence Graphs for BatchNorm:



Convergence Graphs for Dropout:



Convergence Graphs for Weight Decay:

